

# Система автоматизированного обогащения товарных данных для интернет-магазина

Фролов М. А.

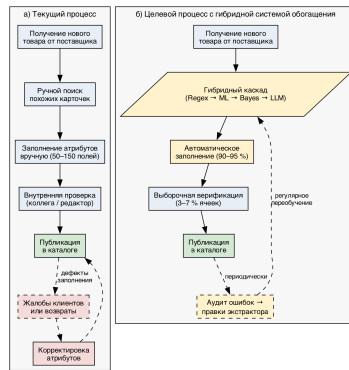
Московский авиационный институт  
Институт № 8, кафедра 806 «Вычислительная математика и программирование»  
Научный руководитель: Судаков В. А.

Москва, 2026

# Ручное обогащение не масштабируется

- ▶ Каталоги маркетплейсов — **десятки миллионов SKU**.
- ▶ Партнёры присылают **лишь базовый набор** полей.
- ▶ РИМ-системы **хранят, но не извлекают** атрибуты.
- ▶ Ручное заполнение — **узкое место** вывода ассортимента.

$\sim 10^7$  SKU · часы на карточку ·  
 линейная стоимость от объёма



Процесс контент-менеджера: текущий → целевой

# Прямой вызов LLM — слишком дорого

## Противоречия

- ▶ **Процесс** —  $10^7+$  SKU vs пропускная способность контент-менеджеров.
- ▶ **Средства** — PIM хранит, но не извлекает атрибуты.
- ▶ **Технология** — точность БЯМ vs её **цена** на промышленных объёмах.

**Задача ВКР.** Программное средство обогащения товарных данных с заданной точностью при **кратном сокращении стоимости** относительно прямого вызова большой языковой модели и допускающее локальное развёртывание.

*⇒ перенос акцента с «одна большая модель на всё» на  
**поатрибутную маршрутизацию** между слоями разной стоимости.*

# Аналоги: пробел в существующих подходах

| Критерий                             | TXtract  | MAVE     | FrugalGPT | AutoMix | Наша система |
|--------------------------------------|----------|----------|-----------|---------|--------------|
| Поатрибутная политика                | ✓        | ✓        | —         | —       | ✓            |
| Калиброванный отказ от ответа        | —        | —        | —         | ✓       | ✓            |
| Маршрутизация по стоимости           | —        | —        | ✓         | ✓       | ✓            |
| Контроль на непересекающихся товарах | —        | —        | —         | —       | ✓            |
| Гибридная архитектура слоёв          | —        | —        | —         | —       | ✓            |
| Локальное развёртывание              | частично | частично | —         | —       | ✓            |

Ни одна из существующих систем не объединяет **поатрибутную политику**, **контроль на новых товарах** и **гибридную архитектуру** одновременно.

*TXtract / MAVE — реализации Amazon / Google: возможен self-hosted, но требует GPU и инфраструктуры BERT-семейства.*

# Цель и задачи ВКР

**Цель.** Программное средство обогащения товарных данных на основе гибридной каскадной архитектуры.

## Частные задачи

1. Разработать архитектуру гибридного каскадного конвейера (5 слоёв) с поатрибутной политикой маршрутизации.
2. Реализовать каждый слой как самостоятельный программный компонент с независимым обновлением.
3. Построить эталонную разметку и тестовую выборку без пересечения товарных кодов (code-disjoint).
4. Провести экспериментальную оценку: точность, покрытие, стоимость; сравнение с прямым вызовом БЯМ.
5. Разработать демонстрационный программный комплекс с REST API и веб-интерфейсом.

# Гипотеза: каскад дешевле без потери точности

**Объект.** Процессы обогащения карточек товаров в интернет-магазинах.

**Предмет.** Методы и алгоритмы извлечения атрибутов, политики маршрутизации между слоями разной стоимости и протоколы их оценки.

**Рабочее предположение.**

- ▶ **Каскад** из слоёв разной стоимости (regex → ML → Bayes → LLM) обеспечивает заданную точность обогащения товарных данных.
- ▶ **Поатрибутная политика эскалации** по калиброванной уверенностикратно сокращает стоимость относительно прямого вызова большой языковой модели.
- ▶ **Локальное развёртывание** возможно на коммерчески доступной инфраструктуре.

# На что опираемся

## 1. Унаследованная инфраструктура

PIM (Akeneo, Pimcore, Salsify) хранит, но не извлекает атрибуты.

## 2. Теоретическая основа

Многоязычные эмбединги, калиброванная классификация (Platt + isotonic), байесовские сети, prompt-инженерия (few-shot, schema-constrained).

## 3. Технологический стек

Python / FastAPI / Go-шлюз / React; Docker-compose; XGBoost, sentence-transformers, pgmpy; Ollama для локальной языковой модели.

## 4. Открытые данные

Open Food Facts (4,5 млн SKU, ~40 % FR / 10 % EN / 7 % DE) + открытые источники по электронике для холодного старта.

# Методы исследования

- ▶ **Системный анализ** — декомпозиция процесса, выявление противоречий
- ▶ **Управление требованиями** — ФР / НФР / А, интегральный критерий Q
- ▶ **Машинное обучение** — эмбединги + XGBoost, калибровка Platt + isotonic
- ▶ **Промпт-инженерия** — few-shot со схемой ответа, селективная эскалация
- ▶ **Предрегистрируемый эксперимент** — McNemar + Бонферрони, Wilson 95 % CI
- ▶ **Итеративная разработка** — цикл аудит → правки → переобучение → измерение

# Основные результаты работы

1. **Требования и качество:** 6 ФР, 5 НФР, 3 А; критерий Q с 4 показателями.
2. **Методика:** 4-слойный каскад с поатрибутной политикой эскалации.
3. **Архитектура:** REST-микросервис, упакованный в `docker-compose`.
4. **Реализация:** ~15 тыс. строк Python, 21 пара «категория-атрибут» (8/6/7 для `pasta/chocolate/cheeses`), работающий e2e демо.
5. **Оценка:** достигнуты целевые показатели качества (точность, сокращение стоимости, покрытие, калибровка) — детали далее.

# Формальная постановка задачи

**Дано.** Товар

$x = (\text{name}, \text{brand}, \text{ingr}, \text{qty}, \text{code})$ ;  
множество атрибутов  $A$ ,  $|A| = 21$  (паста 8 / шоколад 7 / сыры 6); для каждого  $a$  — словарь  $\mathcal{Y}_a$  с отказом  $\perp$ .

**Политика обогащения.**

$$\pi_a = f_4 \circ \sigma_3 \circ f_3 \circ \sigma_2 \circ f_2 \circ \sigma_1 \circ f_1$$

$f_i(x) \rightarrow (\hat{y}, c)$  — слой  $i$  возвращает значение и уверенность;  $\sigma_i$  — ворота: принять  $\hat{y}$  при  $c \geq \tau_{i,a}$ , иначе передать дальше.

**Оптимизационная задача.**

$$\max_{\{\tau_{i,a}\}} \frac{1}{|A|} \sum_a \text{acc}_a^{e2e} \text{ при } \text{cost}_{\text{LLM}} \leq \beta = 10\%.$$

**Ограничения**

- ▶ признаки только из партнёрских полей (без `categories_tags` — анти-утечка);
- ▶ проверка на непересекающихся товарных кодах;
- ▶ локальное развёртывание возможно (слой 4 — open-model).

# Состав требований

## Функциональные (что делает)

- ▶ **F1.** Значение каждого атрибута
- ▶ **F2.** Слой-источник предсказания
- ▶ **F3.** Калиброванная уверенность  
 $c \in [0, 1]$
- ▶ **F4.** Эскалация на запасной слой
- ▶ **F5.** Байес-валидация  
подозрительных решений
- ▶ **F6.** Поатрибутная настройка через  
конфиг

## Нефункциональные (как работает)

- ▶ **N1.** Доля LLM  $\leq 10\%$  ячеек
- ▶ **N2.** Локальное развёртывание (без  
внешних API)
- ▶ **N3.** Обобщение на новых товарных  
кодах
- ▶ **N4.** Воспроизводимость через  
`reproduce.sh`

## Архитектурные (как устроено)

**A1.** `docker-compose`; **A2.** REST-контракт  
между слоями; **A3.** холодный старт без  
правки ядра.

# Критерий качества Q

Качество описано **четырьмя показателями** (точность, стоимость, покрытие, калибровка), сведёнными в интегральный критерий Q:

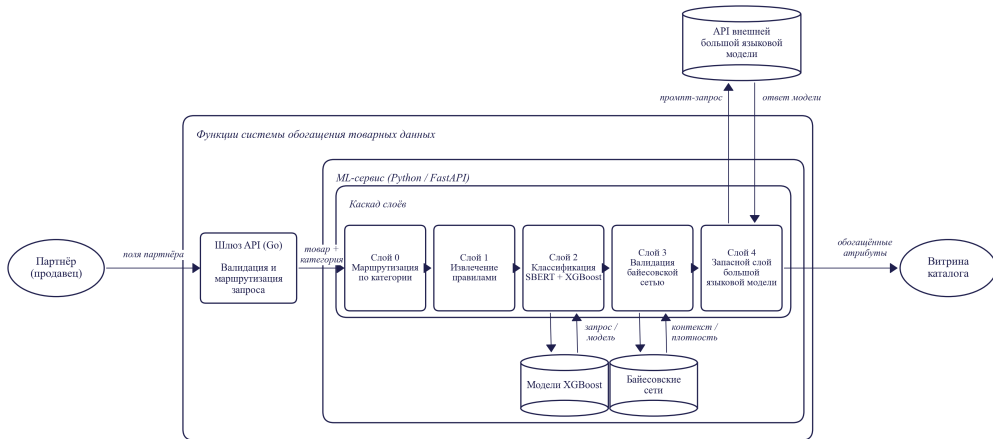
$$Q = w_1 \cdot \text{Acc} + w_2 \cdot \left(1 - \frac{C_{\text{LLM}}}{C_{\text{max}}}\right) + w_3 \cdot \text{Cov} - w_4 \cdot \text{ECE}$$

Acc — точность на непересекающихся товарных кодах (code-disjoint);  $C_{\text{LLM}}/C_{\text{max}}$  — доля LLM-вызовов; Cov — покрытие каскадом до LLM; ECE — ошибка калибровки.

**Веса:**  $w_1 = 0,40$ ,  $w_2 = 0,30$ ,  $w_3 = 0,20$ ,  $w_4 = 0,10$  (приоритет точности над стоимостью согласован с типовыми требованиями РИМ-индустрии).

| Показатель                      | Целевое     | Достигнуто             |
|---------------------------------|-------------|------------------------|
| Точность каскада                | $\geq 90\%$ | 95,5 %                 |
| $C_{\text{LLM}}/C_{\text{max}}$ | $\leq 0,10$ | 0,041                  |
| Cov (покрытие до LLM)           | $\geq 0,90$ | 0,959                  |
| ECE (калибровка)                | $\leq 0,05$ | per-attr ниже целевого |

# Архитектура системы



Каждый слой возвращает  $(\hat{y}, c, \ell)$  или отказ; эскалация — только при низкой уверенности.

# Bayes — валидатор, не классификатор

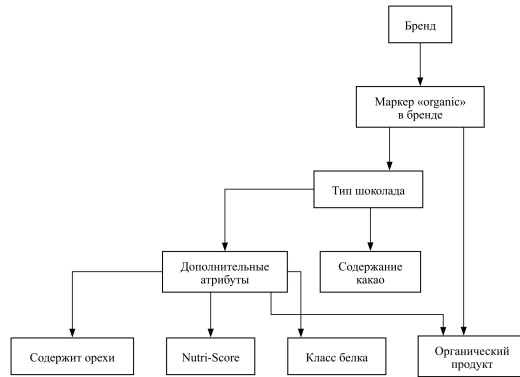
Что построено. pgmpy 1.1.2: структура DAG

— Hill Climb + BIC; CPT —

BayesianEstimator с BDeu.

Почему не классификатор:

- ▶ самостоятельная точность  $\approx 52\%$  (ниже majority baseline),
- ▶ но  $P(\hat{y} | \epsilon)$  на нижнем 5-перцентиле даёт сигнал отбраковки,
- ▶ включён точно на **1 паре из 20** — chocolate/contains\_nuts: **+6,6 пп** точности при **+5,3 пп** расходов на LLM.



DAG для категории chocolate

# Что делает каждый слой

**21 классификатор** по парам «категория-атрибут» (8/6/7); бюджет LLM  $\beta = 10\%$ , фактически **4,1 %**.

| Слой             | Что делает                                          | Когда не справляется                   | Доля          |
|------------------|-----------------------------------------------------|----------------------------------------|---------------|
| <b>L0</b> router | XGBoost на эмбединге → категория + OOD-флаг         | OOD ⇒ слой 4 «как есть»                | —             |
| <b>L1</b> regex  | Поатрибутные шаблоны; точность 100 % на эталоне     | Нет матча ⇒ слой 2                     | 21,9 %        |
| <b>L2</b> ML     | XGBoost на 768-мерном эмбединге; Platt + isotonic   | $p_{\max} < \tau_a \Rightarrow$ слой 3 | <b>73,2 %</b> |
| <b>L3</b> Bayes  | Валидатор плотностей $P_B(\hat{y}   e)$             | $P_B < \theta_a \Rightarrow$ слой 4    | 0,8 %         |
| <b>L4</b> LLM    | gpt-oss-120b / Sonnet 4.5 / Gemini Flash + few-shot | — (последний слой)                     | 4,1 %         |

⇒ 95 % ячеек закрываются дешёвыми слоями (L1+L2+L3); LLM — только на сложном остатке.

## Слой 2: выбор алгоритма

Сравнение четырёх алгоритмов на одной выборке и одинаковом эмбединге (методология §3.3.7.5).

| Классификатор                     | Micro-acc      | Macro-F1     | ECE          | $t_{\text{обуч, с}}$ |
|-----------------------------------|----------------|--------------|--------------|----------------------|
| <b>XGBoost</b> (производственный) | 92,02 %        | 0,849        | <b>0,026</b> | 1062                 |
| MLP (нейронная сеть)              | <b>92,67 %</b> | <b>0,857</b> | 0,031        | 54                   |
| Логистическая регрессия           | 91,85 %        | 0,836        | 0,035        | 8                    |
| Random Forest                     | 89,94 %        | 0,809        | 0,027        | 523                  |

**Почему XGBoost.** MLP даёт чуть выше точность, но **хуже калибровку** (ECE 0,031 vs 0,026) — критично для поатрибутных порогов  $\tau_a$ . RF проигрывает на 2 п. п. по точности.

Логрессия — слишком простая для разнородных категорий.

# Данные и протокол оценки

**Источник.** Open Food Facts, 4,5 млн SKU (~40 % FR / 10 % EN / 7 % DE).

**3 категории.** Паста (8 атр.), шоколад (7), сыры (7) — **21 пара** «категория-атрибут».

**Эталон.** Два уровня: основной LLM-consensus ( $n = 16\,360$ ) + проверочный ручной ( $n = 566$ , нижняя оценка).

**Протокол.** Code-disjoint split; предрегистрация cd9ac7a (2026-05-13).

| Категория              | Атрибутов |
|------------------------|-----------|
| Паста                  | 8         |
| Шоколад                | 7         |
| Сыры                   | 7         |
| <i>Холодный старт:</i> |           |
| Электроника            | 8         |

# Главный результат

| Метрика                               | Значение               | <i>n</i> |
|---------------------------------------|------------------------|----------|
| Точность каскада (микро, CI Вильсона) | 95,5 % [95,2; 95,8]    | 16 360   |
| Макро-F1 каскада (cells-weighted)     | 0,890                  | 16 360   |
| Точность маршрутизатора категории     | 97,2 %                 | 570      |
| Покрытие сквозной цепочки             | 97,3 %                 | 16 360   |
| Точность сквозной цепочки             | 93,0 % [92,6; 93,4]    | 16 360   |
| Нижняя оценка на ручной разметке      | 86,7 % (каскад 91,3 %) | 566      |

Проверка на непересекающихся товарных кодах (code-disjoint). **Доля языковой модели: 4,1 %** ячеек  $\Rightarrow$  сокращение стоимости в **24×** раз.

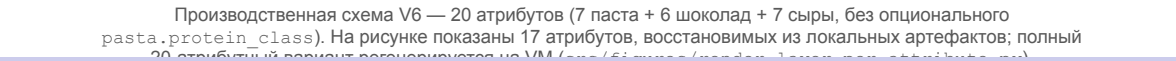
# Каскад vs прямая LLM

Что значит «прямая LLM»: одна и та же модель спрашивается на 100 % ячеек, без слоёв L1/L2/L3. Сравнение на одной и той же выборке (n=16 360 cascade-valid).

| Конфигурация                            | Точность      | Стоимость | Дешевле в   |
|-----------------------------------------|---------------|-----------|-------------|
| Прямая Claude Sonnet 4.5 (опорная)      | 83,8 %        | 1,000     | 1×          |
| Прямая gpt-oss-120b (открытая)          | 69,3 %        | 0,030     | 33×         |
| <b>Каскад + Gemini 2.5 Flash</b>        | <b>93,0 %</b> | 0,0017    | <b>580×</b> |
| Каскад + Claude Sonnet 4.5              | 92,9 %        | 0,041     | 24×         |
| <b>Каскад + gpt-oss-120b (локально)</b> | <b>90,6 %</b> | 0,0012    | <b>810×</b> |

**Ключевые наблюдения:** прямой Sonnet ниже каскада+Gemini Flash на **9,2 п. п.** при стоимости в **580 раз** выше; каскад с малой открытой моделью даёт **+21,3 п. п.** над прямым вызовом той же модели.

■ L1 — regex   ■ L2 — ML (X)



# Соответствие требованиям

1. **Соответствие требованиям.** Все 6 ФР, 5 НФР, 3 А реализованы; **4/4** показателя качества достигли целевых.
2. **Объём.** ~15 тыс. строк Python; 35+ модулей; 7 категориальных схем; 21 пара «категория-атрибут» (8/6/7).
3. **Демо.** `docker-compose`: React + Go + FastAPI; 5 REST-эндпоинтов; ~200 мс на запрос.
4. **Воспроизводимость.** ~70 тестов; ноутбук как исполняемый §3.3; `reproduce.sh` восстанавливает headline-цифры.

# Что нового в работе

## Научная значимость.

- ▶ Методика построения гибридного каскада с **поатрибутной калиброванной уверенностью** и **предрегистрируемым** протоколом оценки.
- ▶ Предрегистрированный **отказ** от гипотезы об обучаемом маршрутизаторе в пользу статической политики (простота и предсказуемость).
- ▶ Воспроизводимость методики подтверждена на новой категории (электроника) в режиме холодного старта.

## Практическая значимость.

- ▶ Работающее ПО в `docker-compose`, интегрируется рядом с PIM.
- ▶ Кратное сокращение стоимости и ручной нагрузки контент-менеджеров.
- ▶ Возможность локального развёртывания — независимость от внешних API.

# Почему числам можно верить

## Достоверность

- ▶ Code-disjoint выборка: основная  $n=16\,360$  + проверочная  $n=566$
- ▶ **Предрегистрация** гипотезы о маршрутизаторе — `git cd9ac7a` до создания артефакта
- ▶ Wilson 95 % CI на всех headline-числах
- ▶ McNemar + поправка Бонферрони ( $\alpha/3$ ) для множественных сравнений
- ▶ Полное воспроизведение через `reproduce.sh`

**Обоснованность.** Признанные методы: sentence-transformers, XGBoost, байесовские сети, калибровка Platt + isotonic; сравнение с TXtract / MAVe / OpenTag / FrugalGPT / AutoMix.

# Демо: ~200 мс на запрос

### Обогащение товарных данных

Каскад: `rorex` → `ML` → `Bayes` → `LLM-fallback`. Введите частично заполненный товар — система предскажет недостающие атрибуты.

#### Ввод товара

Режим определения категории

☒ Авто (классификатор)
 ☐ Вручную

Категория

Шоколад / десерты

Название товара \*

Link Excellence Dark 70% Cocoa 100g

Бренд / производитель

Lindt

Состав

cocoa mass, sugar, cocoa butter, soya lecithin, vanilla extract

Количество

100 g

Примеры:

Patis — Bâlella
  Chocolate — Lindt 70%
  Cheesecake — Request

с произвольными
  White ch. 70% cocoa
  White + gluten-free

Обогатить

#### Результат

|            |                 |                |                   |               |        |
|------------|-----------------|----------------|-------------------|---------------|--------|
| Категория: | chocolate       | Авто (poster): | chocolate, p=0.97 | OOD:          | нет    |
| Закрыто:   | 6 / 7 атрибутов | LLM-fallback:  | 1 атрибут         | Время ответа: | 184 мс |

ПРЕДСКАЗАНИЯ ПО АТТРИБУТАМ

| АТТРИБУТ            | ПРЕДСКАЗАНИЕ        | СЛОЙ  | УВЕРЕННОСТЬ |
|---------------------|---------------------|-------|-------------|
| cocoa_content_class | 70-85 %             | rorex | 1.00        |
| chocolate_type      | dark                | rorex | 1.00        |
| contains_nuts       | false               | ml    | 0.93        |
| is_organic          | false               | ml    | 0.87        |
| contains_milk       | false               | ml    | 0.81        |
| flavor_modifier     | vanilla             | bayes | 0.74        |
| filling_type        | — будет дозаполнено | llm   | —           |

Слой каскада: rorex, ml, bayes, llm-fallback

Рисунок 4.1 — Веб-интерфейс демонстрационной системы (схематично): форма ввода и блок результата с отметкой слоя-источника и оценкой уверенности

Стек **веб-интерфейс** (React) ↔ **шлюз** (Go) ↔ **ml\_service** (FastAPI). Для каждой ячейки — значение, слой-источник и калиброванная уверенность; время ответа ~ 200 мс.

# Внедрение, эксплуатация, развитие

## Внедрение

- ▶ `docker-compose` up рядом с PIM.
- ▶ Стартовый серебряный эталон из истории каталога.
- ▶ Поатрибутные пороги через JSON-конфиги.
- ▶ Языковая модель: внешний API или локальный Ollama.

## Эксплуатация

- ▶ Мониторинг доли языковой модели ( $\leq 10\%$ ).
- ▶ Точность на выборке  $n \geq 200$  еженедельно.
- ▶ Аудит при росте байесовских флагов  $> 5\%$ .
- ▶ Верификация только отмеченных ячеек.

## Развитие

- ▶ Холодный старт на новые категории.
- ▶ Расширение схем атрибутов.
- ▶ Замена базовой модели эмбедингов на более крупную.
- ▶ Подключение более дешёвых языковых моделей.

# Эффект в одном экране

**95,5 %**

точность каскада  
( $n=16\,360$ )

**93,0 %**

точность  
производственной  
цепочки

**4,1 %**

доля ячеек,  
уходящих в языковую  
модель

**24×**

сокращение стоимости  
относительно прямого  
вызова

⇒ кратное сокращение ручной нагрузки и стоимости обогащения  
при сохранении локального развёртывания.

# Заключение

| Задача                      | Результат                                                                                                                                                   |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Архитектура каскада      | 5 слоёв с конъюнктивным AND-условием на пороги; явный отказ слоя; идентификация слоя-источника каждого предсказания.                                        |
| 2. Реализация слоёв         | 21 классификатор по 3 категориям (8/6/7); независимое обновление; docker-compose: Go-шлюз + Python/FastAPI ML-сервис + Web-UI.                              |
| 3. Эталонная разметка       | LLM-consensus gold $n=16\,360$ + human gold $n=688$ в-scope (566 cascade-valid); code-disjoint split (brand overlap 82–85 %).                               |
| 4. Экспериментальная оценка | точность каскада 95,5 % / 0,890 макро-F1, производственная 93,0 %, человеческий gold 86,7 %; сокращение стоимости 24× архитектурное / 580× комбинированное. |
| 5. Демо-комплекс            | docker-compose (Go + FastAPI + Web-UI); REST-контракт; 3 сценария Layer 4 (Sonnet 4.5 / Gemini Flash / open gpt-oss-120b).                                  |

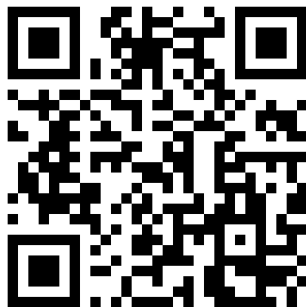
Итого: точность каскада 95,5 % / макро-F1 0,890, производственная оценка 93,0 %, человеческий gold 86,7 %, сокращение стоимости 24× архитектурное / 580× комбинированное.

# Спасибо за внимание

Точность каскада **95,5 %**,  
производственной цепочки **93,0 %**,  
сокращение стоимости в **24×** раз.

Исходный код, ноутбук §3.3 в исполняемом  
виде, демо-комплекс и текст работы — в  
репозитории.

## Вопросы?



[github.com/Qworl/diploma](https://github.com/Qworl/diploma)

## B0. Метрики и обозначения (для вопросов)

**AND-условие слоя 2.** Предсказание  $\hat{y}$  принимается

$\iff P_{\text{ML}}(\hat{y} \mid x) \geq \tau_a \wedge P_B(\hat{y} \mid e) \geq \theta_a$  — формализация ансамблевой оценки уверенности как конъюнкции порогов ML-классификатора ( $\tau_a$ ) и байесовского валидатора ( $\theta_a$ ).

**Метрики:**

- ▶  $\text{acc}_k^{\text{cov}} = \frac{|\hat{y}_k = y_k \wedge \hat{y}_k \neq \text{null}|}{|\hat{y}_k \neq \text{null}|}$  — точность на покрытой части (только по ячейкам, где  $\hat{y} \neq \perp$ );
- ▶  $\text{cov}_k = \frac{|\hat{y}_k \neq \text{null}|}{N}$  — покрытие, доля ячеек с ненулевым предсказанием;
- ▶  $\text{acc}_k^{\text{e2e}} = \text{acc}_k^{\text{cov}} \cdot \text{cov}_k$  — сквозная точность (отказ  $\perp$  засчитывается как ошибка), production-realistic;
- ▶  $\text{cost}_{\text{LLM}}$  — ожидаемая доля вызовов запасного слоя на 1000 ячеек (бюджет  $\beta = 10\%$ ).

*Связь  $\text{acc}^{\text{e2e}} = \text{acc}^{\text{cov}} \cdot \text{cov}$  объясняет разрыв между только-каскадной точностью (95,5 %) и сквозной с маршрутизатором (93,0 %): часть ячеек теряется на  $\text{cov} < 1$ .*

# В1. Отрицательный результат: обучаемый маршрутизатор

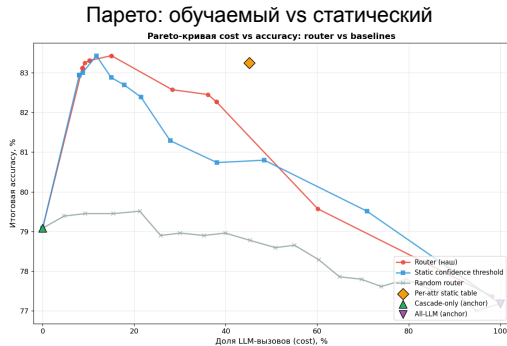
**Гипотеза Н1:** обучаемый стоимостно-чувствительный маршрутизатор превосходит статическое правило хотя бы на одном из трёх бюджетов языковой модели (25 % / 40 % / 50 %).

**Предрегистрация:** git cd9ac7a, 2026-05-13 02:11 (за 1 ч 11 мин до создания артефакта).

**Поправка Бонферрони:**  $\alpha/3 \approx 0,0167$ .

**Минимально различимый эффект (MDE) при  $n = 1539$ :**  $\approx 4,4$  пп (парный McNemar).

**Результат:**  $p_{\text{McNemar}} = 0,74/0,49/1,00$  для бюджетов 25 / 40 / 50 % (все  $> \alpha/3$ ).

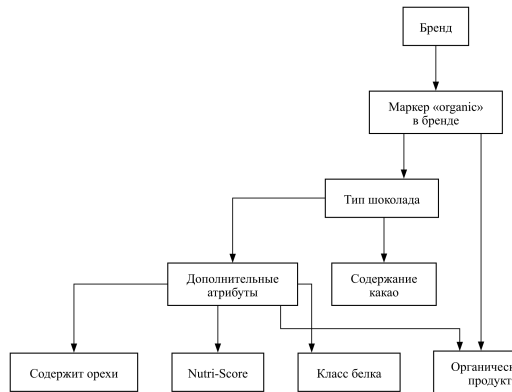


## В2. Байесовский слой как валидатор плотностей

**Что построено.** BN на Hill Climb + BIC (silver-train, code-disjoint); CPT доучены на CONCAT(silver, gold-train×10) с BDeu.

**Классификатор отвергнут.** Точность на сложном остатке после L2 —  $\approx 52\%$ , ниже majority. Переориентирован в валидатор:  $P(\hat{y} | e)$  ниже 5-го перцентиля  $\Rightarrow$  эскалация на L4.

**Включение точечное.** 1 из 20 пар: chocolate/contains\_nuts — +6,6 пп точности при +5,3 пп расходов. На остальных 19 селективность сигнала  $\leq$  базовой ошибки.



## В3. Серебряный сигнал vs контрольная разметка

**Серебряная разметка** (консенсус тегов OFF + 3 языковые модели + слепая разметка Orpus) служит **обучающим** сигналом. Контрольная выборка получена по аудит-циклу.

**Цикл «аудит — правки экстрактора — переобучение — измерение»** воспроизведён для всех 3 категорий с приростом **+18,2, +16,3, +14,4** пп.

**Калибровка уверенности** — комбинация Platt + изотонической регрессии поатрибутно; ошибка ECE измеряется на отдельной выборке.

## В4. Утечки и борьба с ними

**Запрет утечки categories\_tags в эмбединги.**

Эмбединги Слой 2 строятся **только** из партнёрских полей: product\_name + brands + ingredients\_text + quantity.categories\_tags — это **выход** обогащения, не вход; использование как признака привело бы к утечке целевой переменной.

**Разбиение по непересекающимся товарным кодам (60/20/20; brand overlap train↔eval 82–85%).** Бренды в обучающей, валидационной и контрольной выборках не пересекаются — гарантия обобщения на новые бренды.

**Анализ по расстоянию k-NN (§3.3.7.3):** точность **не падает** на удалённых от обучающей выборки товарах — монотонный рост от Q1 (82,7 %) до Q4 (91,6 %).

## B5. Многоязычность

**Языковой состав OFF:**  $\approx 40\%$  FR,  $10\%$  EN,  $7\%$  DE, остальное — прочие.

**Эмбединг:** `paraphrase-multilingual-mpnet-base-v2` (50+ языков, архитектура MPNet, общее семантическое пространство).

**Поатрибутно-поязыковой срез** ( $n=3257$  ячеек, предыдущая выборка до extended rerun 2026-05-27 — пересчёт перенесён в перспективу развития темы): взвешенная точность стабильна по основным языкам.

**Под балансировку каскад не оптимизировался** — это естественный выигрыш многоязычной базовой модели эмбедингов.

## В6. Стоимость языковой модели на 1 млн SKU

**Каскад делегирует языковой модели лишь 4,1 % ячеек** ( $n = 17\,062$ ) — прямая прокси к стоимости. Сокращение затрат **95,9 % (в  $24\times$  раз)** относительно «прямой языковой модели на всё».

**Распределение по слоям:** L1 (regex высокой точности) — 21,9 %, L2 (ML) — **73,2 %**, L3 (regex низкой точности) — 0,8 %, L4 (языковая модель) — 4,1 %.

**Проекция на 1 млн SKU  $\times$  20 атрибутов** ( $\sim 2,0 \cdot 10^7$  ячеек):

- ▶ Прямая модель — оплачены все  $\sim 2,0 \cdot 10^7$  вызовов.
- ▶ Каскад — оплачены лишь  $\sim 8,2 \cdot 10^5$  вызовов (4,1 %).

**Топ-3 затратных атрибутов для языковой модели:** pasta.grain\_type (32,9 %), pasta.pasta\_shape (22,8 %), cheeses.is\_pdo (13,6 %).

## В7. Холодный старт: электроника (§5 ноутбука)

Демонстрация переноса методики на **новую категорию без регулярных правил**.

**Электроника (8 атрибутов):** смартфоны — RAM, объём памяти, диагональ экрана, поддержка 5G, ...

**Перенос:** активны только Слой 2 (ML + эмбединги) и Слой 4 (языковая модель); Слой 1 пропускается; Слой 3 — по обучающим парам.

**Результат:** холодный старт работает на ограниченной обучающей выборке, что подтверждает применимость каскада за пределами продуктового домена.

## В8. Где каскад слабее: precision/recall на 3 худших атрибутах

Производственная схема V6, эталон LLM-consensus (extended rerun, n=16 360).

| Атрибут                                                                                      | Класс                         | n   | prec   | recall        | F1    |
|----------------------------------------------------------------------------------------------|-------------------------------|-----|--------|---------------|-------|
| <i>chocolate.flavor_profile</i> — <i>micro</i> 87,5 %, <i>macro-F1</i> 0,684, <i>n</i> =1060 |                               |     |        |               |       |
|                                                                                              | intense_bitter                | 397 | 91,1 % | 98,0 %        | 0,944 |
|                                                                                              | nutty                         | 281 | 86,6 % | 85,4 %        | 0,860 |
|                                                                                              | sweet_creamy                  | 190 | 80,8 % | 84,2 %        | 0,825 |
|                                                                                              | <i>salty_caramel</i> (редкий) | 56  | 90,5 % | <b>67,9 %</b> | 0,776 |
| <i>pasta.is_filled</i> — <i>micro</i> 88,5 %, <i>macro-F1</i> 0,786, <i>n</i> =1225          |                               |     |        |               |       |
|                                                                                              | false (не фаршированные)      | 958 | 87,2 % | 100 %         | 0,931 |
|                                                                                              | <i>true</i> (фаршированные)   | 267 | 100 %  | <b>47,2 %</b> | 0,641 |
| <i>cheeses.texture</i> — <i>micro</i> 89,0 %, <i>n</i> =335                                  |                               |     |        |               |       |
|                                                                                              | hard                          | 109 | 95,5 % | 96,3 %        | 0,959 |
|                                                                                              | soft                          | 97  | 91,3 % | 97,9 %        | 0,945 |
|                                                                                              | <i>cream</i> (редкий)         | 33  | 73,9 % | <b>51,5 %</b> | 0,607 |

## B9. chocolate/contains\_nuts — атрибут с активным Bayes-валидатором

Единственная пара из 20, где Bayes-валидатор включён в production (см. B2).  
Структура ошибок после фильтрации:

| Класс              | n    | precision     | recall         | F1    |
|--------------------|------|---------------|----------------|-------|
| false (без орехов) | 749  | <b>99,1 %</b> | 93,2 %         | 0,961 |
| true (с орехами)   | 377  | 87,9 %        | <b>98,4 %</b>  | 0,929 |
| <b>Сумма</b>       | 1126 | micro 94,9 %  | macro-F1 0,945 |       |

**Почему именно Bayes-валидатор.** Атрибут безопасный (аллергия — наличие орехов важно подтверждать жёстко). Распределение асимметричное: false-предсказания *критичны*, true-предсказания допускают перепроверку. Bayes-плотность ловит сомнительные false и эскалирует их в LLM — **+6,6 п. п.** категориальной точности chocolate при **+5,3 п. п.** расходов на LLM.

## B10. Macro-F1 на ручной разметке: 0,843 cells-weighted

**Консервативная нижняя оценка** на независимом эталоне ручной разметки (Claude Opus 4, n=688 в-scope, n=566 cascade-valid).

| Метрика                            | LLM-consensus (n=16 360) | Ручной gold (n=566) |
|------------------------------------|--------------------------|---------------------|
| Cascade micro-acc                  | 95,5 %                   | 91,3 %              |
| Cascade macro-F1 (cells-weighted)  | 0,890                    | <b>0,843</b>        |
| Cascade macro-F1 (attr-unweighted) | 0,892                    | 0,833               |
| Сквозная (производственная) acc    | 93,0 %                   | 86,7 %              |

**Разрыв 4,2 п. п.** на cascade micro-acc и **0,047** на macro-F1 объясняется тремя причинами (детали в §3.3 ВКР):

- ▶ ручной gold — независимый сигнал, не коррелирует с обучающим silver,
- ▶ меньший  $n$  (566 vs 16 360) даёт более широкий доверительный интервал,